UP LT GRADE · COMPUTER SCIENCE MAINS

Database Management Systems — Complete Descriptive-Exam Notes

Simple, Clear, Exam-Ready — With Diagrams

Foundations: Database Systems & Data Models · Database Languages · DBMS Architecture · Users & Data Independence

Modeling & Design: ER Modeling & Diagrams · Relational Model · SQL · Normalization

Systems Aspects: Database Security · Transaction Management · Query Processing & Optimization

Reliability: Concurrency Control · Recovery Techniques

Prepared for descriptive-exam revision · 45-day study plan

Table of Contents

1. Database Systems, View of Data & Data Models	Page 3
2. Database Languages (DDL, DML, DCL, TCL)	Page 5
3. DBMS Architecture (Three-Schema Architecture)	Page 6
4. Database Users & Data Independence	Page 8
5. ER Modeling — Entities, Attributes & Relationships	Page 9
6. ER Diagram Notation & Worked Example	Page 11
7. Relational Model — Keys & Constraints	Page 13
8. Introduction to SQL	Page 15
9. Relational Database Design — Functional Dependencies & Normalization	Page 17
10. Database Security	Page 19
11. Transaction Management — ACID & States	Page 20
12. Query Processing & Query Optimization	Page 22
13. Concurrency Control	Page 24
14. Recovery Techniques	Page 26

FOUNDATIONS

1. Database Systems, View of Data & Data Models

> **In Simple Words**
>
> A Database System stores and manages data so many users/programs can access it reliably, without each one needing to know exactly how the data is physically stored. A Data Model is the conceptual toolset used to describe the structure of that data.

Why Not Just Use Files?

- Traditional file processing systems suffer from Data Redundancy, Inconsistency, difficulty in Data Access, Isolation, Integrity problems, Atomicity issues, Concurrent access anomalies, and Security problems.
- A DBMS (Database Management System) solves these by centralizing data, enforcing integrity rules, controlling redundancy, and providing concurrent, secure, atomic access.

View of Data — Data Abstraction

DBMS hides complexity from users through three levels of abstraction (elaborated fully as the Three-Schema Architecture in Topic 3):

- **Physical Level:** Describes HOW data is actually stored (bytes, blocks, indexes).
- **Logical Level:** Describes WHAT data is stored and the relationships among it (tables, columns).
- **View Level:** Describes only PART of the database relevant to a specific user, hiding the rest.

Data Models

Model	Description
Hierarchical Model	Data organized as a tree — parent-child relationships only (one parent per child).
Network Model	Data organized as a graph — a child can have multiple parents, more flexible than hierarchical.
Relational Model	Data organized as tables (relations) with rows and columns — the dominant model today, based on set theory.
Entity-Relationship (ER) Model	Conceptual model using entities, attributes, and relationships — mainly used for database DESIGN before implementation.
Object-Oriented Model	Data represented as objects (like in OOP), combining data and the methods that operate on it.

> **Exam Tip**
>
> When asked to compare data models, always mention the STRUCTURE they use (tree/graph/table/objects) as the key differentiator — this single detail anchors the entire comparison.

FOUNDATIONS

2. Database Languages (DDL, DML, DCL, TCL)

> **In Simple Words**
>
> A DBMS provides different "sub-languages" for different jobs — defining the structure of data, manipulating the data itself, controlling access/permissions, and managing transactions. In practice, all of these are part of SQL.

The Four Categories

Language	Full Form	Purpose	Example Commands
DDL	Data Definition Language	Defines/modifies database structure (schema)	CREATE, ALTER, DROP, TRUNCATE
DML	Data Manipulation Language	Inserts, updates, retrieves, deletes actual data	SELECT, INSERT, UPDATE, DELETE
DCL	Data Control Language	Controls access rights and permissions	GRANT, REVOKE
TCL	Transaction Control Language	Manages transactions	COMMIT, ROLLBACK, SAVEPOINT

DML: Procedural vs Declarative

- **Procedural DML:** User specifies WHAT data is needed AND HOW to get it (step-by-step) — e.g., relational algebra.
- **Declarative (Non-Procedural) DML:** User specifies only WHAT data is needed, not how to retrieve it — the DBMS figures out the "how" internally. SQL is primarily declarative — e.g., relational calculus.

> **Exam Tip**
>
> Always give at least 2 example commands per category (DDL/DML/DCL/TCL) — examiners frequently ask "classify the following commands," so memorizing the command-to-category mapping is directly scoring.

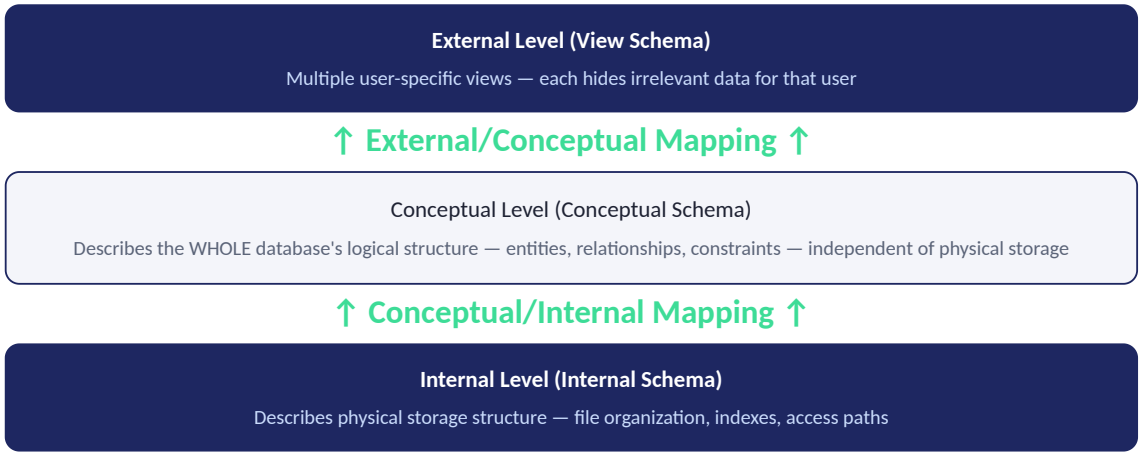
FOUNDATIONS

3. DBMS Architecture (Three-Schema Architecture)

In Simple Words

The Three-Schema Architecture separates the user's view of data from the physical storage details, achieved through three distinct layers, each describing the database at a different level of abstraction.

Diagram: Three-Schema Architecture



Purpose of Each Level

- **External Level:** Closest to users; different users/applications can have different views (e.g., a billing app sees only payment fields).
- **Conceptual Level:** The "middle" schema, describing what data exists and how it relates, without concern for physical storage or user-specific views.
- **Internal Level:** Closest to physical storage; deals with actual file structures, block sizes, and indexing on disk.

Exam Tip

Always draw this three-layer diagram (as shown above) when asked about "DBMS architecture" — it's THE most frequently asked diagram question in DBMS papers, and directly connects to the Data Independence topic that follows.

FOUNDATIONS

4. Database Users & Data Independence

In Simple Words

Different types of people interact with a database in different ways, and "Data Independence" means you can change one schema level without needing to change the levels above it.

Types of Database Users

- **Database Administrators (DBA):** Manage the overall database — schema definition, access control, performance tuning, backup/recovery.
- **Naive/End Users:** Interact via pre-built application interfaces (e.g., ATM users) without knowing SQL or database internals.
- **Application Programmers:** Write application programs (using host languages + embedded DML) that interact with the database.
- **Sophisticated Users:** Interact directly with the database using query languages (SQL) without writing full application programs — e.g., data analysts.

Data Independence — Two Types

Type	Meaning
Logical Data Independence	Ability to change the CONCEPTUAL schema without changing EXTERNAL schemas/application programs. Harder to achieve, since applications are closely tied to logical structure.
Physical Data Independence	Ability to change the INTERNAL schema (storage/indexing) without changing the CONCEPTUAL schema. Easier to achieve, since it's "lower down" and more isolated.

Key Connection

Data Independence is possible BECAUSE of the Three-Schema Architecture (Topic 3) — each level is insulated from the one below via mappings, so a change at one level only requires updating that level's mapping, not rewriting every level above.

Exam Tip

Always explicitly state WHICH schema changes and WHICH schema stays unaffected for each type of data independence — this precise cause-and-effect pairing is exactly what examiners check for.

MODELING & DESIGN

5. ER Modeling — Entities, Attributes & Relationships

> **In Simple Words**
>
> The Entity-Relationship (ER) Model is a conceptual blueprint for a database, built from three basic building blocks: Entities (things), Attributes (their properties), and Relationships (how entities connect to each other).

Core Building Blocks

- **Entity:** A real-world object or concept that can be distinctly identified — e.g., a Student, a Course. An Entity Set is a collection of similar entities.
- **Attribute:** A property that describes an entity — e.g., Student has Name, RollNo, Age.
- **Relationship:** An association between two or more entities — e.g., a Student ENROLLS IN a Course.

Types of Attributes

- **Simple vs Composite:** Simple cannot be divided further (Age); Composite can be broken into sub-parts (Name → First Name + Last Name).
- **Single-valued vs Multi-valued:** Single-valued holds one value (DOB); Multi-valued can hold several (PhoneNumbers).
- **Stored vs Derived:** Stored is directly recorded (DOB); Derived is calculated from another attribute (Age, derived from DOB).
- **Key Attribute:** Uniquely identifies each entity in the entity set (e.g., RollNo).

Degree and Cardinality of Relationships

- **Degree:** Number of entity sets participating — Unary (1), Binary (2, most common), Ternary (3).
- **Cardinality Ratios (for binary relationships):**
  - **One-to-One (1:1):** One entity of set A relates to exactly one entity of set B.
  - **One-to-Many (1:N):** One entity of set A relates to many entities of set B.
  - **Many-to-Many (M:N):** Many entities of set A relate to many entities of set B.

Strong vs Weak Entity Sets

- **Strong Entity Set:** Has its own key attribute sufficient to uniquely identify its entities (e.g., Student with RollNo).
- **Weak Entity Set:** Does NOT have a sufficient key of its own; it depends on a Strong (owner) entity set and uses a Partial Key combined with the owner's key — e.g., a "Dependent" of an Employee, identified only in combination with the Employee's ID.

> **Exam Tip**
>
> When explaining weak entities, always name the "Identifying/Owner Relationship" that connects it to its strong entity, and mention the Partial Key explicitly — both terms are commonly checked for by examiners.

MODELING & DESIGN

6. ER Diagram Notation & Worked Example

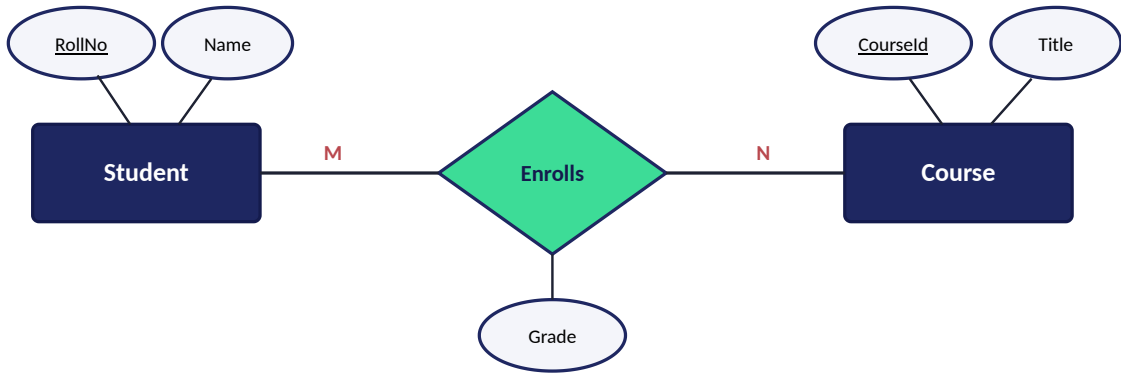
In Simple Words

An ER Diagram is the visual drawing of the ER Model using standard symbols: rectangles for entities, ovals for attributes, diamonds for relationships, and lines connecting them with cardinality labels.

Standard ER Notation

Symbol	Represents
Rectangle	Entity Set
Double Rectangle	Weak Entity Set
Oval/Ellipse	Attribute
Underlined Oval	Key Attribute
Dashed Oval	Derived Attribute
Double Oval	Multi-valued Attribute
Diamond	Relationship Set
Double Diamond	Identifying Relationship (for weak entity)
Lines with 1 / N labels	Cardinality of the relationship

Worked Example: Student – Enrolls – Course (Many-to-Many)



Reading This Diagram

Student (key: RollNo) and Course (key: CourseId) are both strong entities. The "Enrolls" relationship connects them with M:N cardinality — one Student can enroll in many Courses, and one Course can have many Students. "Grade" is a descriptive attribute of the RELATIONSHIP itself (not of either entity alone), since a grade only makes sense in the context of one specific student-course enrollment.

Exam Tip

When asked to "draw an ER diagram for..." a given scenario, always: (1) identify entities first, (2) identify their key attributes, (3) identify relationships and their cardinality, (4) only then add non-key descriptive attributes — this order prevents missing a key element under time pressure.

MODELING & DESIGN

7. Relational Model — Keys & Constraints

In Simple Words

The Relational Model represents data as Relations (tables) — a table's rows are called Tuples, and columns are called Attributes. Keys and Constraints ensure the data stored is unique, valid, and consistent.

Types of Keys

Key	Definition
Super Key	Any set of attributes that can uniquely identify a tuple (may contain extra, unnecessary attributes).
Candidate Key	A MINIMAL super key — no attribute can be removed without losing uniqueness. A table can have multiple candidate keys.
Primary Key	The candidate key CHOSEN by the designer to uniquely identify tuples; cannot be NULL.
Alternate Key	Candidate keys NOT chosen as the primary key.
Foreign Key	An attribute in one table that refers to the Primary Key of another (or the same) table, enforcing referential integrity.

Integrity Constraints

- **Domain Constraint:** Each attribute's value must come from its defined domain (data type/range).
- **Entity Integrity Constraint:** No attribute of the PRIMARY KEY can be NULL.
- **Referential Integrity Constraint:** A foreign key value must either match an existing primary key value in the referenced table, or be NULL.
- **Key Constraint:** No two tuples can have the same value for the primary key.

Relational Algebra — Core Operations

Operation	Symbol	Purpose
Select	σ	Selects rows satisfying a condition
Project	π	Selects specific columns
Union	∪	Combines tuples from two compatible relations
Set Difference	−	Tuples in one relation but not another
Cartesian Product	×	Combines every tuple of one relation with every tuple of another
Join (Natural Join)	⋈	Combines related tuples from two relations based on a common attribute

Exam Tip

Always give the exact definitions of Super Key vs Candidate Key vs Primary Key together as a set (as above) — this is one of the most frequently repeated "differentiate" questions, and the minimality property of Candidate Key is the detail most students miss.

MODELING & DESIGN

8. Introduction to SQL

In Simple Words

SQL (Structured Query Language) is the standard language used to define, manipulate, and query relational databases — combining DDL, DML, DCL, and TCL commands into one language.

Basic SQL Query Structure

SELECT-FROM-WHERE Structure

```
SELECT column1, column2 FROM table_name WHERE condition;
```

This maps directly to relational algebra: SELECT clause = Projection (π), WHERE clause = Selection (σ), FROM clause = the relation(s) being queried.

Common SQL Clauses

- **WHERE:** Filters rows based on a condition.
- **GROUP BY:** Groups rows sharing a common value, typically used with aggregate functions.
- **HAVING:** Filters GROUPS (applied after GROUP BY) — unlike WHERE, which filters individual rows before grouping.
- **ORDER BY:** Sorts the result set (ASC or DESC).
- **JOIN:** Combines rows from two or more tables based on a related column.

Aggregate Functions

Function	Purpose
COUNT()	Number of rows/values
SUM()	Total of numeric values
AVG()	Average of numeric values
MAX() / MIN()	Highest / lowest value

Types of JOINs

- **INNER JOIN:** Returns only matching rows from both tables.
- **LEFT (OUTER) JOIN:** Returns all rows from the left table, plus matches from the right (NULLs where no match).
- **RIGHT (OUTER) JOIN:** Returns all rows from the right table, plus matches from the left.
- **FULL (OUTER) JOIN:** Returns all rows from both tables, with NULLs where there's no match on either side.

Solved Example

Find department-wise average salary for departments with more than 5 employees:

```
SELECT dept_id, AVG(salary) FROM employees GROUP BY dept_id HAVING COUNT(*) > 5;
```

Note: WHERE cannot be used with COUNT(*) here because grouping hasn't happened yet at the WHERE stage — this is exactly why HAVING exists.

Exam Tip

When writing SQL queries in the exam, always follow correct clause ORDER (SELECT…FROM…WHERE…GROUP BY…HAVING…ORDER BY) and terminate with a semicolon — syntax precision is directly graded, not just logical correctness.

MODELING & DESIGN

9. Relational Database Design — Functional Dependencies & Normalization

In Simple Words

Normalization is the process of organizing tables to reduce redundancy and avoid update/insert/delete anomalies, guided by rules called Functional Dependencies (FDs) — which describe how one attribute determines another.

Functional Dependency (FD)

Definition

X → Y means: given the value of attribute set X, the value of Y is uniquely determined. X is called the Determinant.
Example: RollNo → Name (knowing RollNo uniquely determines the Name).

Normal Forms

Normal Form	Rule
1NF	Every attribute must hold only ATOMIC (indivisible) values — no repeating groups or multi-valued attributes in a single cell.
2NF	Must be in 1NF, AND no Partial Dependency — every non-key attribute must depend on the WHOLE primary key, not just part of it (relevant only for composite keys).
3NF	Must be in 2NF, AND no Transitive Dependency — a non-key attribute must not depend on another non-key attribute.
BCNF (Boyce-Codd NF)	Stricter version of 3NF — for every FD X→Y, X must be a super key (removes remaining anomalies that 3NF can miss).

Solved Example — Detecting a Partial Dependency (violates 2NF)

Table: Enrollment(StudentId, CourseId, StudentName, Grade), Primary Key = (StudentId, CourseId).
FD: StudentId → StudentName. Since StudentName depends on ONLY StudentId (part of the composite key), not the whole key (StudentId, CourseId), this is a **Partial Dependency** — violates 2NF. Fix: split into Student(StudentId, StudentName) and Enrollment(StudentId, CourseId, Grade).

Why Normalize?

- **Update Anomaly:** Same fact stored in multiple rows — updating one place but missing another causes inconsistency.
- **Insertion Anomaly:** Cannot insert certain data without also having unrelated data available (e.g., can't add a course with no students enrolled yet, if course info is stuck inside the Enrollment table).
- **Deletion Anomaly:** Deleting one fact accidentally deletes other, still-needed information.

Exam Tip

For normalization questions, always name the SPECIFIC dependency type causing the violation (partial/transitive) and show the decomposed tables explicitly — vague answers without naming the dependency type lose marks even if the final tables are correct.

SYSTEMS ASPECTS

10. Database Security

In Simple Words

Database Security protects data from unauthorized access, modification, or destruction, using a combination of authentication, authorization, and encryption mechanisms.

Key Security Mechanisms

- **Authentication:** Verifying WHO is trying to access the database (username/password, biometrics, tokens).
- **Authorization (Access Control):** Determining WHAT an authenticated user is allowed to do — implemented via GRANT/REVOKE privileges (SELECT, INSERT, UPDATE, DELETE) on specific tables/views.
- **Views as a Security Tool:** A view can expose only specific columns/rows to a user, hiding sensitive data without duplicating the underlying table.
- **Encryption:** Protects data both at rest (stored) and in transit (network), so even if intercepted, it's unreadable without the decryption key.
- **Audit Trails:** Logging who accessed/modified what data and when, for accountability and forensic analysis.

SQL Access Control Example

Example Commands

`GRANT SELECT, INSERT ON employees TO user_ravi;` — gives Ravi permission to read and add rows.
`REVOKE INSERT ON employees FROM user_ravi;` — removes only the insert privilege, SELECT remains.

SQL Injection (Common Threat)

A security attack where malicious SQL code is inserted into input fields, tricking the database into executing unintended commands. Prevented using parameterized queries/prepared statements instead of directly concatenating user input into SQL strings.

Exam Tip

If asked about database security "measures," structure the answer using the mechanism list above (Authentication → Authorization → Views → Encryption → Auditing) — this progression from "who" to "what" to "how protected" reads as a complete, structured answer.

SYSTEMS ASPECTS

11. Transaction Management — ACID Properties & States

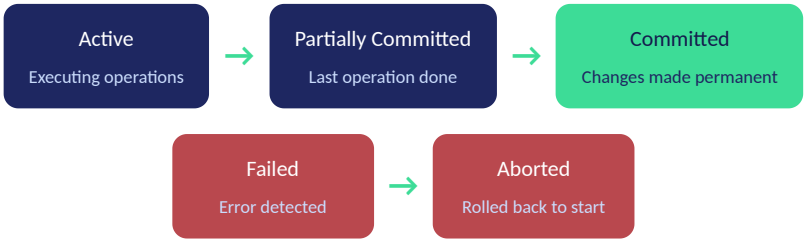
In Simple Words

A Transaction is a single logical unit of work (e.g., transferring money between accounts) that must execute completely or not at all. ACID properties guarantee transactions behave reliably even with failures or concurrent access.

ACID Properties

Property	Guarantee
Atomicity	A transaction is "all or nothing" — either ALL its operations complete, or NONE do (rolled back on failure).
Consistency	A transaction takes the database from one valid state to another, preserving all defined integrity rules.
Isolation	Concurrent transactions execute as if they ran one at a time (serially), even though they may actually interleave.
Durability	Once a transaction commits, its changes persist even if the system crashes immediately after.

Diagram: Transaction State Diagram



Note: "Active" can also transition directly to "Failed" if an error is detected mid-execution, not just from "Partially Committed."

Exam Tip

Always draw BOTH the state diagram AND explain all four ACID properties with a one-line real example each (e.g., Atomicity: bank transfer either fully completes or doesn't happen at all) — combining diagram and example answers is what separates 8/8 from 5/8.

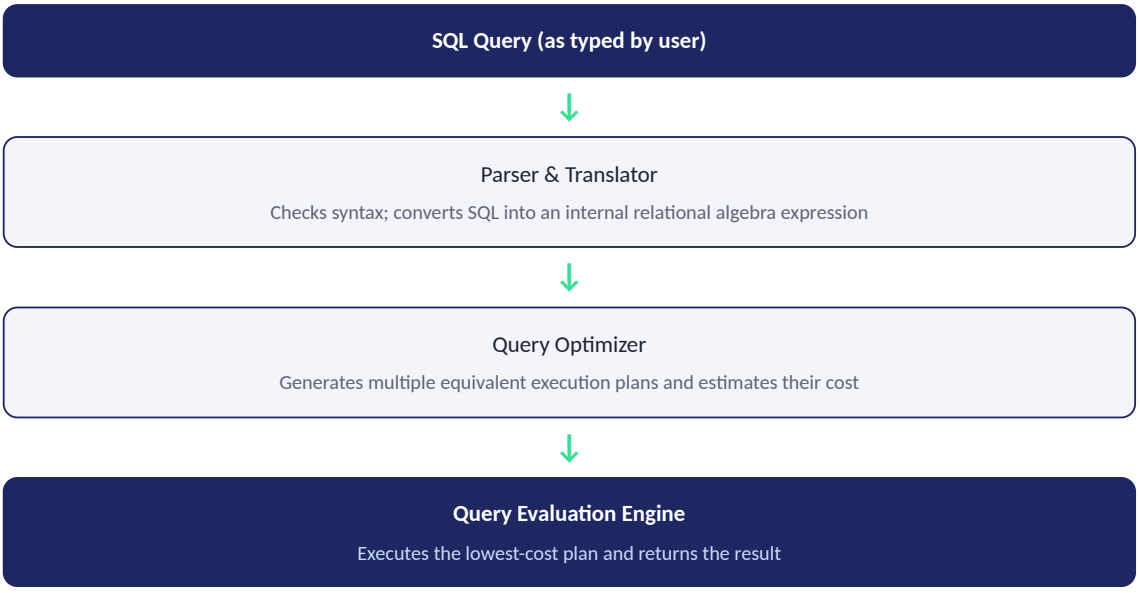
SYSTEMS ASPECTS

12. Query Processing & Query Optimization

In Simple Words

When you submit an SQL query, the DBMS doesn't run it exactly as written — it translates, analyzes, and rewrites it into the most EFFICIENT execution plan before actually running it. This entire pipeline is called Query Processing, and choosing the best plan is Query Optimization.

Diagram: Query Processing Pipeline



Query Optimization Approaches

- **Heuristic (Rule-Based) Optimization:** Applies proven transformation rules — e.g., "perform SELECT/Selection operations as early as possible" (reduces the number of tuples processed by later steps), "perform PROJECT before JOIN where possible" (reduces the number of columns carried forward).
- **Cost-Based Optimization:** Estimates the actual COST (I/O operations, CPU time) of different execution plans using statistics (table size, index availability) and picks the cheapest one.

Why Optimize?

Key Insight

The SAME SQL query can be executed in many different ways (different join orders, different access paths) that all produce the identical correct RESULT, but with drastically different COST. Query Optimization doesn't change correctness — it only changes efficiency.

Exam Tip

Always draw the Parser → Optimizer → Evaluation Engine pipeline (as above) and give at least ONE specific heuristic rule (like "push selection down") — abstract explanations without a concrete rule example score lower.

RELIABILITY

13. Concurrency Control

In Simple Words

When multiple transactions run at the same time on the same data, Concurrency Control mechanisms prevent them from interfering with each other in ways that would violate the Isolation property (part of ACID).

Problems Without Concurrency Control

- **Lost Update Problem:** Two transactions both read a value, both update it independently — one update overwrites (and loses) the other.
- **Dirty Read Problem:** A transaction reads data written by another transaction that hasn't yet committed — if that transaction later rolls back, the first one used invalid data.
- **Unrepeatable Read Problem:** A transaction reads the same row twice and gets DIFFERENT values because another transaction modified it in between.

Lock-Based Concurrency Control

- **Shared Lock (S):** Allows multiple transactions to READ the same data item simultaneously, but no one can write while any shared lock is held.
- **Exclusive Lock (X):** Allows only ONE transaction to both read and write the data item — no other transaction can hold any lock on it at the same time.
- **Two-Phase Locking (2PL):** Each transaction has a Growing Phase (only acquiring locks, never releasing) followed by a Shrinking Phase (only releasing locks, never acquiring) — guarantees serializability.

Timestamp-Based Concurrency Control

Core Idea

Every transaction gets a unique Timestamp when it starts. Conflicting operations are ordered strictly by timestamp — an older transaction is always treated as having "come first," avoiding the need for locks entirely, at the cost of possibly aborting/restarting younger transactions that conflict.

Deadlock in Concurrency Control

Since locking can cause transactions to wait for each other's locks, Deadlock (covered in the OS Concurrency topic) can occur here too — handled via the same Prevention/Avoidance/Detection strategies, or by using timestamp ordering to sidestep locking altogether.

Exam Tip

When asked to explain Two-Phase Locking, always explicitly name BOTH phases (Growing and Shrinking) and state that lock RELEASE never happens before ALL locks are acquired — this exact wording is what confirms understanding to an examiner.

RELIABILITY

14. Recovery Techniques

In Simple Words

Recovery Techniques ensure that after a system crash or failure, the database can be restored to a consistent state — preserving the Durability and Atomicity properties even when things go wrong mid-transaction.

Log-Based Recovery

- The DBMS maintains a Log — a sequential record of every database modification, written to stable storage BEFORE the actual data is modified (Write-Ahead Logging, WAL).
- Each log record typically contains: Transaction ID, Data item, Old value, New value.
- On recovery after a crash, the log is used to either REDO committed transactions or UNDO uncommitted ones.

Recovery Using Log Records

Technique	When Used
UNDO	Applied to transactions that had started but NOT committed before the crash — their partial changes are reversed using the "old value" in the log.
REDO	Applied to transactions that HAD committed before the crash but whose changes might not have reached disk yet — reapplied using the "new value" in the log.

Checkpoints

Why Checkpoints Matter

Without checkpoints, recovery would need to scan the ENTIRE log from the very beginning of time. A Checkpoint periodically records that all transactions committed before that point have been safely written to disk — recovery then only needs to process the log from the MOST RECENT checkpoint onward, dramatically reducing recovery time.

Shadow Paging (Alternative to Logging)

Instead of logging changes, this technique maintains two page tables — a Current Page Table (being modified) and a Shadow Page Table (the last consistent, committed version). On commit, the current table becomes the new shadow table; on crash, the system simply reverts to the shadow table, discarding all uncommitted changes instantly. Avoids the need for UNDO/REDO, but has overhead in maintaining and copying page tables.

Exam Tip

Always explain WAL (Write-Ahead Logging) as the foundational principle before describing UNDO/REDO — examiners specifically check whether you understand that the LOG must be written before the DATA, not after, since that ordering is what makes recovery possible at all.

— End of Notes —

Database Management Systems | UP LT Grade CS Mains Preparation